Deliverable #2 Template

SE 3A04: Software Design II – Large System Design

Tutorial Number: T01

Group Number: G06

Group Members:

- Aakash Satishkumar
- Ali Shareeff
- Harmanpreet Singh Sagar
- Jason Kim
- Praneet Singh

IMPORTANT NOTES

Please document any non-standard notations that you may have used

- *Rule of Thumb*: if you feel there is any doubt surrounding the meaning of your notations, document them

Some diagrams may be difficult to fit into one page

- Ensure that the text is readable when printed, or when viewed at 100% on a regular laptop-sized screen.
- If you need to break a diagram onto multiple pages, please adopt a system of doing so and thoroughly explain how it can be reconnected from one page to the next; if you are unsure about this, please ask about it

Please submit the latest version of Deliverable 1 with Deliverable 2

- Indicate any changes you made.

If you do NOT have a Division of Labour sheet, your deliverable will NOT be marked

1 Introduction

This section should provide a brief overview of the entire document.

1.1 Purpose

This document provides a high-level overview of the SCEMAS system architecture. It describes the structure of the system, its major subsystems, and the key design decisions that guide how the system is organized and implemented. It also outlines how the different components of the system interact and the responsibilities of each subsystem.

The intended audience is any relevant SCEMAS stakeholder. This includes but is not limited to the general public, government officials, external developers, city operators, and SCEMAS team members.

1.2 System Description

SCEMAS is a system that uses sensors to gather environmental data, such as temperature, air quality, and noise levels, and transforms it into valuable information that stakeholders can use. The system identifies abnormal values or threshold violations, and record alerts when potential issues are detected. It maintains both real-time and historical environmental data, allowing trends and anomalies to be analyzed over time. SCEMAS also manages validated data and makes it available through a public API while keeping track of system events and alerts.

1.3 Overview

Section 2 presents the Analysis Class Diagram for SCEMAS, showing the key classes and their relationships within the system. Section 3 describes the architectural design of SCEMAS, including the chosen architectural styles, subsystem structure, and reasoning behind the design decisions. Finally, Section 4 provides Class Responsibility Collaboration (CRC) Cards, detailing the responsibilities of each class and how they interact with other components in the system.

2 Analysis Class Diagram

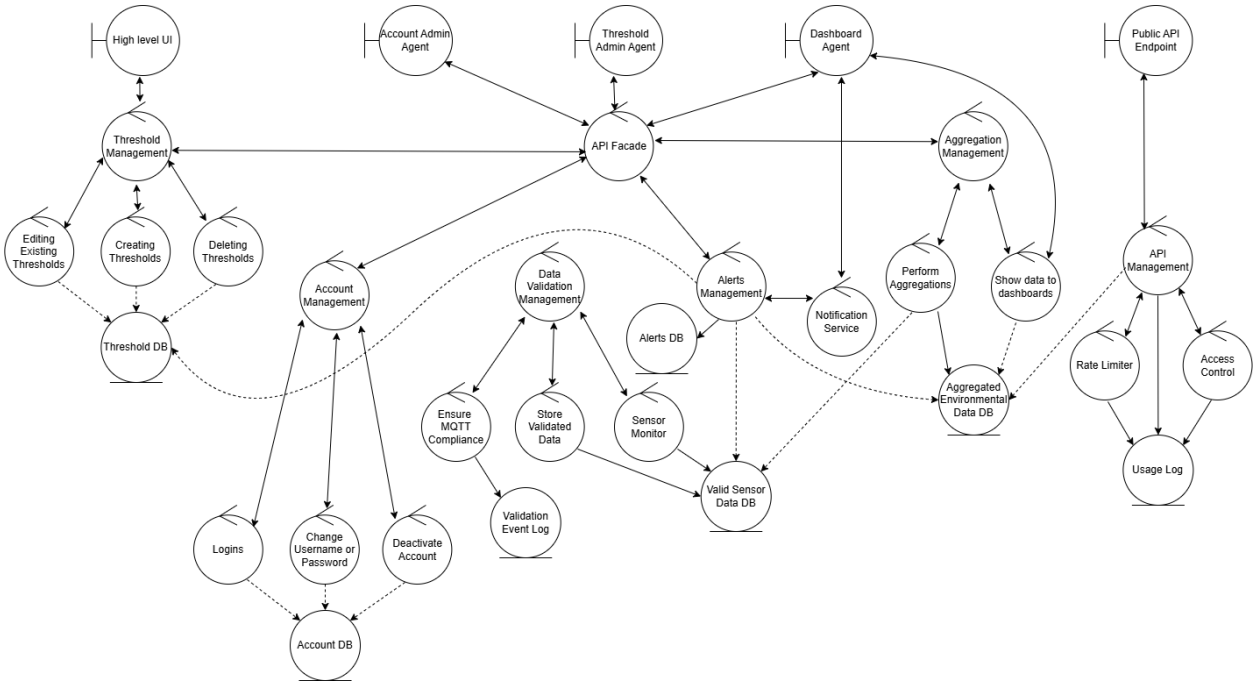

Figure 1: Analysis Class Diagram

3 Architectural Design

This section should provide an overview of the overall architectural design of your application. Your overall architecture should show the division of the system into subsystems with high cohesion and low coupling.

3.1 System Architecture

SCEMAS uses a mix of architectures that combines PAC in the presentation layer with Repository backend subsystems. The user interface is structured as a one PAC unit, where the Presentation contains the different SCEMAS views, the Abstraction maintains the UI's current state, and the Control coordinates user actions. The Control communicates with the backend using an API Facade, which provides one consistent entry point for the UI and routes requests to the correct subsystem. SCEMAS is broken down into backend subsystems that implement the core system behavior and persist system state through a repository layer implemented across multiple databases, each dedicated to specific data.

Table 1: Subsystem Architecture Table

Subsystem	Purpose	Architecture Style
Account Management	Handles authentication, sessions, and role-based access for operators and administrators.	Repository
Threshold Management	Allows authorized users to create, edit, delete, and store threshold rules for alert evaluation.	Repository
Alerts Management	Checks validated readings against thresholds, detects anomalous trends, creates alerts, and tracks alert history and status changes.	Repository
API Management	Serves public read-only requests, validates inputs, applies rate limiting and blocking, and records usage for monitoring.	Repository
Data Validation	Receives MQTT telemetry, validates format and ranges, and stores trusted readings for downstream processing.	Repository
Aggregation Management	Computes anomalous trends, zone-level summaries like averages and maximums, and stores results for dashboards, alerts, and the public API.	Repository

SCEMAS uses a hybrid architecture approach because the system must simultaneously support many interactive dashboards, using real-time data collection and processing, while being reliant on centralized data storage.

PAC architecture is used on the overall presentation layer of the system to structure the user interface into cohesive, independent agents that each contain a presentation, abstraction, and control component to ensure interactivity within the overall system. This architecture is ideal for the overall system structure because SCEMAS will contain many user interfaces, each with different responsibilities. Separating each interface will introduce modularity and allow each agent to independently update without affecting each other. This is particularly helpful in SCEMAS as the various types of user interfaces available within the system serve different stakeholders but must interact within the same system environment.

The Repository architecture style is used by all subsystems. This is because these components primarily serve to perform consistent, structured operations on persistent data. A centralized repository allows for multiple subsystems to access consistent data within the system environment. This architecture style also serves to improve data integrity and simply data sharing between subsystems, simultaneously avoiding duplicate data storage. This architecture style also improves modularity and maintainability, allowing each subsystem to remain highly cohesive as each focus on one specific function of the system.

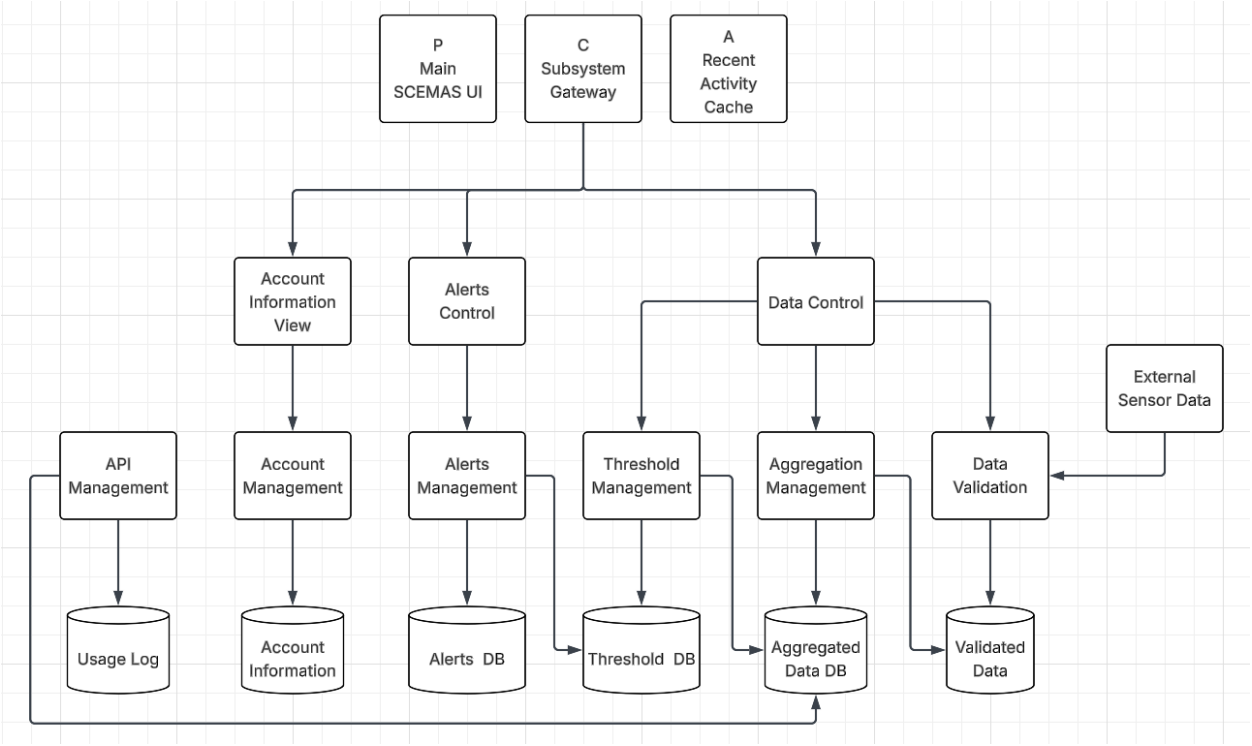

Figure 2: System Architecture Diagram

Blackboard architecture for parts of the system was considered because multiple components consume telemetry data. However, this choice was eliminated because SCEMAS

does not involve multiple independent "Knowledge Sources" solving an open-ended problem together. Instead, the system follows a clear processing flow where the system ingests, validates, stores, aggregates, and evaluates thresholds which are better represented by repository-based subsystems.

MVC was also considered to be the main overall architecture, but it was eliminated because the user interface is better represented as multiple independent views with their own control and state. PAC models this more directly by separating each UI agent into its own Presentation, Abstraction, and Control components, reducing the coupling between the different UIs.

A repository only system architecture was not chosen because the UI does not interact directly with the database. The UI agents talk to the backend through a shared API Facade, which sends each request to the right backend service and keeps the UI from needing to know the internal logic or database details.

Using PAC beyond the UI layer was eliminated as PAC is most useful for structuring interactive interfaces. For backend processing and persistence (validation, aggregation, alerting), enforcing PAC would add extra layers and complexity without improving maintainability.

3.2 Subsystems

Our system has been subdivided into six separate subsystems: Account Management, Threshold Management, Alerts Management, API Management, Data Validation, and Aggregation Management.

The Account Management subsystem is responsible for handling user authentication, identity management, and account lifecycles for users of SCEMAS, namely creation and deactivation. This subsystem will focus on users and administrators to securely log into the system and manage their account, reflecting changes in a secured persistent database. It will enforce RBAC, ensuring that only authorized users of a certain access level can perform specific actions.

The Threshold Management subsystem is responsible for defining and maintaining environmental thresholds that determine whether alerts or actions should be triggered. This subsystem allows authorized users to create new thresholds by specifying parameters, limits, and associated conditions. Existing thresholds can be edited to reflect updated requirements, and deprecated thresholds can be deleted when they are no longer needed. A higher-level user interface is provided to facilitate the management of these thresholds in a structured manner. This subsystem directly supports the Alerts Management subsystem by supplying the criteria used to evaluate sensor data.

The Alerts Management subsystem is responsible for detecting when environmental conditions violate defined thresholds and ensuring that operators are notified. It uses the thresholds defined in Threshold Management to evaluate validated sensor readings and generate an alert whenever a rule is violated. Furthermore, it also utilizes the data

aggregated by Aggregation Management to generate an alert whenever an anomalous trend is detected. Each alert is assigned to a severity level and corresponding message explaining what the alert means. Alerts are stored and tracked through their lifecycle so that they can be reviewed in the future. Finally, the subsystem communicates alert updates to the operational dashboard, so operators can quickly view the issue and act appropriately.

The API Management subsystem handles public, read-only API requests and ensures that access to SCEMAS data is secure and reliable. It receives incoming requests, validates request parameters, and retrieves the requested aggregated environmental information from the Aggregation Management database. The subsystem enforces usage controls like rate limiting to prevent abuse and can block specific IP addresses that continuously violate access policies. It also records API usage for monitoring and auditing. Overall, the API Management subsystem provides external developers with access to environmental data while protecting the systems' availability.

The Data Validation subsystem is responsible for making sure that environmental data collected by the sensors is reasonable, consistent with historical trends, and follows the required protocols before it is used in the system. It processes incoming sensor data to determine if the sensors are working as intended. It also ensures this data complies with the MQTT protocol. Furthermore, this subsystem logs all validation events. It then stores validated data in the time-series database. Overall, the data validation system helps maintain the reliability of SCEMAS data.

The Aggregation Management subsystem is responsible for processing validated environmental sensor data and generating aggregated environmental metrics. It retrieves validated sensor readings from the Valid Sensor Data DB, which serves as part of the system's central repository, and performs aggregation operations such as calculating averages or other summaries across defined time intervals or geographic zones. The results are then stored in the Aggregated Environmental Data DB so they can be efficiently accessed by other subsystems. These aggregated values are used by the Dashboard Agent to display environmental information on operational dashboards and by the API Management subsystem to provide aggregated environmental data to external users. Overall, this subsystem transforms raw validated sensor data into meaningful summaries that support monitoring, analysis, and decision-making across the system.

4 Class Responsibility Collaboration (CRC) Cards

Class Name: Threshold Management (Controller)	
Responsibility:	**Collaborators:**
Knows Creating Thresholds Knows Editing Existing Thresholds Knows Deleting Thresholds Knows High-level UI Knows API Facade	Creating Thresholds Editing Existing Thresholds Deleting Thresholds High-level UI API Facade

Class Name: Creating Thresholds (Controller)	
Responsibility:	**Collaborators:**
Knows Threshold Management Knows Threshold DB Handles process of creating thresholds	Threshold Management Threshold DB

Class Name: Editing Existing Thresholds (Controller)	
Responsibility:	**Collaborators:**
Knows Threshold Management Knows Threshold DB Handles process of editing existing thresholds	Threshold Management Threshold DB

Class Name: Deleting Thresholds (Controller)	
Responsibility:	**Collaborators:**
Knows Threshold Management Knows Threshold DB Handles threshold deletion processes	Threshold Management Threshold DB

Class Name: High-level UI (Boundary)	
Responsibility:	**Collaborators:**
Knows Threshold Management Handles display of UI for modifying thresholds	Threshold Management

Class Name: Threshold DB (Entity)	
Responsibility:	**Collaborators:**
Knows Editing Existing Thresholds Knows Creating Thresholds Knows Deleting Thresholds Knows Alerts Management Responsible for storing thresholds	Editing Existing Thresholds Creating Thresholds Deleting Thresholds Alerts Management

Class Name: API Facade (Controller)	
Responsibility:	**Collaborators:**
Knows Account Admin Agent Knows Threshold Admin Agent Knows Dashboard Agent Knows Threshold Management Knows Account Management Knows Alerts Management Knows Aggregation Management Acts as a central gateway between internal system and external interfaces Routes requests to appropriate subsystem controller	Account Admin Agent Threshold Admin Agent Dashboard Agent Threshold Management Account Management Alerts Management Aggregation Management

Class Name: Account Admin Agent (Boundary)	
Responsibility:	**Collaborators:**
Knows API Facade Responsible for initiating account management operations	API Facade

Class Name: Threshold Admin Agent (Boundary)	
Responsibility:	**Collaborators:**
Knows API Facade Responsible for initiating threshold configuration operations	API Facade

Class Name: Dashboard Agent (Boundary)	
Responsibility:	**Collaborators:**
Knows API Facade Responsible for displaying system data and alerts to operational and public dashboards	API Facade

Class Name: Alerts Management (Controller)	
Responsibility:	**Collaborators:**
Knows Threshold DB Knows Aggregated Environmental Data DB Knows Alerts DB Knows Notification Service Knows API Facade Knows Valid Sensor Data DB Gets active threshold rules from Threshold DB and use them to evaluate incoming validated sensor readings Creates alerts when a threshold is violated, including assigning a severity level and creating a clear message Updates and tracks the alert lifecycle status so they can be reviewed later Publishes alert raised/updated messages so the Dashboard Agent can refresh its view through the shared API Facade.	Threshold DB Aggregated Environmental Data DB Alerts DB Notification Service API Facade Valid Sensor Data DB

Class Name: Alerts DB (Entity)	
Responsibility:	**Collaborators:**
Knows Alerts Management Stores alert details so alerts and their status history persist over time Provides alert data needed for operational monitoring (active alerts and historical alerts)	Alerts Management

Class Name: Notification Service (Controller)	
Responsibility:	**Collaborators:**
Knows Alerts Management Knows Dashboard Agent Delivers alert updates created by Alerts Management to the Dashboard Agent so that users can view and respond quickly Sends notifications when an alert is created or when its status changes	Alerts Management Dashboard Agent

Class Name: API Management (Controller)	
Responsibility:	**Collaborators:**
Knows Aggregated Environmental Data DB Knows Rate Limiter Knows Usage Log Knows Access Control Knows Public API Endpoint Receives and handles incoming public, read-only API requests for aggregated environmental data Validates requests and reject invalid or unsupported requests Enforces API usage controls like rate limiting and blocking unauthorized clients	Aggregated Environmental Data DB Rate Limiter Access Control Usage Log Public API Endpoint

Class Name: Public API Endpoint (Boundary)	
Responsibility:	**Collaborators:**
Knows API Management Calls/Routes to API Management to process requests and return correct response	API Management

Class Name: Rate Limiter (Controller)	
Responsibility:	**Collaborators:**
Knows API Management Knows Usage Log Tracks request counts for each client within a period Allows or denies requests based on configured limits for rate limiting	API Management Usage Log

Class Name: Access Control (Controller)	
Responsibility:	**Collaborators:**
Knows API Management Knows Usage Log Responsible for enforcing role-based access control by checking if a user is authorized to perform the requested action Denies unauthorized requests	API Management Usage Log

Class Name: Usage Log (Entity)	
Responsibility:	**Collaborators:**
Knows API Management Knows Rate Limiter Knows Access Control Responsible for recording API request metadata (timestamp, request, client) for monitoring Supports rate limiting by providing history of API activity	API Management Rate Limiter Access Control

Class Name: Data Validation Management (Controller)	
Responsibility:	**Collaborators:**
Knows Ensure MQTT Compliance Knows Sensor Monitor Knows Store Validated Data Responsible for ensuring data follows MQTT protocol check if sensors are working properly and store validated data in time-series database	Ensure MQTT compliance Sensor Monitor Store Validated Data

Class Name: Ensure MQTT Compliance (Controller)	
Responsibility:	**Collaborators:**
Knows Data Validation Management Knows Validation Event Log Knows how to parse and validate incoming data packets against MQTT protocols and log validation events	Data Validation Management Validation Event Log

Class Name: Sensor Monitor (Controller)	
Responsibility:	**Collaborators:**
Knows Data Validation Management Knows Valid Sensor Data DB Responsible for monitoring incoming data from sensors and detect possible abnormal data by comparing historical trends	Data Validation Management Valid Sensor Data DB

Class Name: Store Validated Data (Controller)	
Responsibility:	**Collaborators:**
Knows Data Validation Management Knows Valid Sensor Data DB Knows how to send data to time series database	Data Validation Management Valid Sensor Data DB

Class Name: Validation Event Log (Entity)	
Responsibility:	**Collaborators:**
Knows Ensure MQTT Compliance Knows how to log protocol validation events and attach timestamps to stored data	Ensure MQTT Compliance

Class Name: Valid Sensor Data DB (Entity)	
Responsibility:	**Collaborators:**
Knows Sensor Monitor Knows Store Validated Data Knows Alerts Management Knows Perform Aggregations Stores validated sensor data in database	Sensor Monitor Store Validated Data Alerts Management Perform Aggregations

Class Name: Account Management (Controller)	
Responsibility:	**Collaborators:**
Knows Logins Knows Change Username or Password Knows Deactivate Account Knows API Facade	Logins Username and password changes Deactivation manager API Facade

Class Name: Logins (Controller)	
Responsibility:	**Collaborators:**
Knows Account Management Knows Account DB Manages user intent to login with the system and validates credentials	Account Management Account DB

Class Name: Change Username or Password (Controller)	
Responsibility:	**Collaborators:**
Knows Account Management Knows Account DB Edit user fields relating to username/password	Account Management Account DB

Class Name: Deactivate Account (Controller)	
Responsibility:	**Collaborators:**
Knows Account Management Knows Account DB Allows users to deactivate accounts	Account Management Account DB

Class Name: Account DB (Entity)	
Responsibility:	**Collaborators:**
Knows Logins Knows Change Username or Password Knows Deactivate Account Responsible for storing user account information	Logins Change Username or Password Deactivate Account

Class Name: Aggregation Management (Controller)	
Responsibility:	**Collaborators:**
Knows Perform Aggregations Knows Show data to dashboards Knows API Facade	Perform Aggregations Show data to dashboards API Facade

Class Name: Show data to dashboards (Controller)	
Responsibility:	**Collaborators:**
Knows Aggregation Management Knows Dashboard Agent Knows Aggregated Environmental Data DB Responsible for sending data to public and operational dashboards for display	Aggregation Management Dashboard Agent Aggregated Environmental Data DB

Class Name: Perform Aggregations (Controller)	
Responsibility:	**Collaborators:**
Knows Aggregation Management Knows Aggregated Environmental Data DB Processes the interaction of calculating aggregated environmental metrics from sensor data	Aggregation Management Aggregated Environmental Data DB

Class Name: Aggregated Environmental Data DB	
Responsibility:	**Collaborators:**
Knows Perform Aggregations Knows Show data to dashboards Knows Alerts Management Knows API Management Responsible for storing aggregated environmental data	Perform Aggregations Show data to dashboards Alerts Management API Management

A Division of Labour

Include a Division of Labour sheet which indicates the contributions of each team member. This sheet must be signed by all team members.

Aakash Satishkumar:

- 3.1
- 3.2 Alert Management, API Management
- 4 Alert Management, API Management Component CRC Cards

Ali Shareeff:

- 1.1
- 1.2
- 3.2 Data Validation
- 4. Data Validation Subsystem Component CRC Cards

Harmanpreet Singh Sagar

- 2 Analysis Class Diagram
- 3.2 Threshold Management
- 4 Threshold Management, API Facade Component CRC Cards
- 4 Modified Aggregation Management component CRC Cards
- 4 Modified Account Management component CRC Cards
- 4 Modified Alerts Management component CRC Cards
- 4 Modified API Management component CRC Cards
- Document Formatting

Jason Kim:

- 3.1
- Structure Architecture Diagram
- 3.2 Account Management
- 4 Account Management Component CRC Cards
- Document Formatting

Praneet Singh

- 1.3
- 3.2 Aggregation Management Component CRC Cards
- 4 Aggregation Management Component CRC Cards